



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING

SECTION 2

INDIVIDUAL PROJECT ON APACHE SPARK

LECTURER: DR. ARYATI BINTI BAKRI

NAME	MATRIC NUMBER
NURUL IKA SYAFINY BINTI AZHAR	A23CS0164

An Automated Multi-Source Credit Card Fraud Detection Pipeline Using Apache Spark and Delta Lake

Nurul Ika Syafiny Binti Azhar
Faculty of Computing
Universiti Teknologi Malaysia
Johor Bahru, Johor
nurulikasyafiny@graduate.utm.my

Abstract— This paper details an end-to-end credit card fraud detection pipeline developed using Apache Spark 4.1.1 and a Delta Lake Data Lakehouse architecture to replace slow, legacy batch processing with low-latency analytical screening. A multi-source Extract-Load-Transform (ELT) Medallion pipeline ingested and harmonized 8,008,974 total transactions, combining 1,296,675 Kaggle records, 284,807 ULB records, and 6,362,620 PaySim records onto a unified schema. Localized memory constraints were bypassed by replacing slow Python User-Defined Functions with native JVM expressions to compute spatial-temporal features, followed by minority up-sampling to balance class distributions. A PySpark MLib Random Forest Classifier achieved a weighted Precision of 99.18%, Recall of 99.21%, F1-Score of 98.96%, and an AUC-ROC of 95.30%. The final automated scoring layer processed the entire serving population, routing exactly 305 high-confidence alerts (probability ≥ 0.75) into an active queue rendered via a Streamlit dashboard. Programmatic validation audits successfully confirmed data flow completeness and table integrity across all processing layers, demonstrating a scalable, mathematically sound architecture for real-time risk mitigation.

Keywords- credit card fraud detection; Apache Spark; data pipeline; Delta Lake; Random Forest; data lakehouse; PySpark; Streamlit; ELT; machine learning

I. INTRODUCTION (HEADING 1)

This project operates within the Finance and E-Commerce domain, specifically addressing the challenge of credit card transaction fraud detection in online retail environments.

A. Problem Statement

With the rapidly growing e-commerce platforms processing thousands of multi-state credit card transactions every hour, companies are increasingly faced with financial losses and customer complaints due to identity theft and fraudulent purchases [3]. Currently, many organizations rely on legacy database systems that execute slow, daily batch queries to flag

suspicious transactions. This means fraud is typically only detected after the products have already been shipped, resulting in direct revenue losses, high chargeback rates, and eroded customer trust.

The core problem is the absence of a timely, automated, and scalable data pipeline capable of ingesting and analyzing high-volume transactional data to detect fraudulent behavior before order fulfillment occurs.

B. Justification

The relevance and urgency of this problem are supported by alarming global statistics. In 2023, global losses from online payment and e-commerce fraud were estimated at USD 48 billion, with payment-card fraud alone accounting for approximately USD 33.8 billion [6], [11]. The U.S. Federal Trade Commission (FTC) Consumer Sentinel Network Data Book (2024) reported that credit card fraud topped the list of identity theft types with over 449,000 reports filed in that year alone [5]. As transaction volumes continue to scale, a reactive, batch-based approach is no longer sufficient. An automated, real-time or near real-time pipeline using distributed computing technology such as Apache Spark is necessary to address the speed, volume, and variety challenges inherent in modern financial transaction data.

C. Objectives

The main goal of this project is to develop an end-to-end automated data pipeline using Apache Spark that can ingest, process, and analyze multi-source credit card transaction data to detect fraudulent transactions in near real-time, thereby enabling proactive fraud intervention before order fulfillment.

The specific objectives of this project are as follows:

1. To design and implement a multi-source data ingestion pipeline using Apache Spark that integrates structured and semi-structured credit card transaction datasets from heterogeneous sources into a unified schema.
2. To apply data cleaning, transformation, and feature engineering techniques on the integrated dataset to prepare it for machine learning-based fraud classification using Apache Spark MLlib.
3. To store the processed and analyzed data in a Data Lakehouse architecture (Delta Lake) and evaluate the performance of the fraud detection model in terms of accuracy, precision, recall, and F1-score.

II. LITERATURE REVIEW

This section reviews relevant literature in two parts: first establishing the business and domain context of credit card fraud, then examining the technical and technological approaches used to detect it, before identifying the gap this project aims to address.

A. Overview of Credit and Debit Card Payments

A credit card is a payment instrument issued by a financial institution that allows a cardholder to borrow funds, up to a pre-approved limit, to pay for goods and services, with the balance repaid later either in full or over time with interest. A debit card, by contrast, draws funds directly from the cardholder's existing bank account at the time of purchase [12]. Both instruments operate over a standard four-party payment network architecture involving the cardholder, merchant, acquiring bank, and issuing bank [14]. Card payments have become the dominant mode of retail transaction globally, credit and debit cards together accounted for approximately 50% of all point-of-sale transactions worldwide in 2023, with credit cards alone representing 27% of transaction volume [13]. This widespread adoption, combined with the rapid growth of e-commerce and card-not-present (CNP) transactions, has substantially expanded the attack surface available to fraudsters, as transactions can now be completed without the cardholder physically presenting the card or being verified in person [15].

B. Credit Card Fraud and How It Occurs

Credit card fraud refers to the unauthorized use of a person's credit or debit card, or the underlying card account information, to obtain money, goods, or services without the cardholder's consent [12]. Fraud of this nature can be broadly categorized into application fraud, where a fraudster uses stolen or fabricated identity information to open a new credit account in someone else's name, and behavioral fraud, where an existing legitimate account is compromised and misused [16]. Behavioral fraud is further divided into several common subtypes: lost or stolen card fraud, where a physical card is used after being taken from the rightful owner; counterfeit or cloned card fraud, where card data captured via skimming devices is copied onto a blank or duplicate card; mail interception fraud, where a newly issued card is intercepted before reaching the

cardholder; and card-not-present (CNP) fraud, where stolen card details are used to complete a transaction online, by phone, or by mail without the physical card being involved [12], [16].

CNP fraud is widely recognized as the fastest-growing and most difficult fraud type to detect, precisely because the merchant has no way to physically verify the card or the identity of the person making the purchase [15]. Fraudsters obtain the underlying card data through several mechanisms, including phishing which is tricking victims into voluntarily disclosing card details, skimming which when fraudsters use a concealed device on an ATM or POS terminal to capture data from the card's magnetic stripe, large-scale data breaches of merchant or payment processor databases, and the purchase of stolen card data, known as "dumps" or "fullz," on dark web marketplaces [15]. Once obtained, fraudsters frequently test the validity of stolen card numbers using small, low-value transactions across multiple merchants before attempting larger purchases, a technique known as "carding" [15], which produces the rapid, multi-merchant, multi-state transaction patterns this project's chosen primary dataset is specifically designed to simulate.

C. Impact of Credit Card Fraud on Businesses and Consumers

The financial and operational impact of credit card fraud on e-commerce businesses is substantial and multifaceted. When a fraudulent transaction is confirmed, the merchant typically bears the direct cost of the chargeback, the value of the shipped goods or rendered services, and an additional chargeback processing fee levied by the payment processor, regardless of whether the merchant acted in good faith [17]. Beyond the direct monetary loss, merchants face reputational damage, increased scrutiny from card networks which may revoke merchant processing privileges if chargeback rates exceed acceptable thresholds, and reduced customer trust, all of which can compound into long-term revenue decline [17]. For consumers, the impact includes the inconvenience and anxiety of disputing fraudulent charges, potential damage to their credit standing if application fraud occurs in their name, and a general erosion of confidence in digital payment channels [12].

D. Automated Data Pipeline in Fraud Detection

As transaction volumes continue to scale, a reactive, batch-based approach to fraud detection is no longer sufficient to address the business problem described above. A data pipeline is a series of automated processes that moves data from one or more sources to a destination, typically performing transformations along the way; in the context of data engineering, pipelines are the backbone of modern analytics and machine learning systems. A data pipeline for financial fraud detection must encompass stream processing capabilities to handle high-velocity transactional data with minimal latency, distinguishing it from conventional batch-oriented data workflows [2]. A pipeline designed purely around batch processing introduces unacceptable detection delays in fraud contexts, where even a few hours of lag can result in completed fraudulent transactions that are difficult to reverse [2], directly

mirroring the legacy-system limitation identified in the business problem statement above.

Pipelines are typically classified by their processing paradigm: batch processing, where data is collected over a period and processed in bulk at scheduled intervals, and stream processing, where data is ingested and processed continuously as it arrives [2]. In the context of e-commerce fraud detection, stream or near real-time processing is strongly preferred, as the window of opportunity to intervene before a fraudulent order is fulfilled is extremely narrow. Additionally, pipelines follow either an ETL (Extract, Transform, Load) or ELT (Extract, Load, Transform) pattern. In a Data Lakehouse architecture, where raw data is stored in a distributed file system before transformation, the ELT pattern is more appropriate, as it allows transformation to occur in-place on a distributed engine such as Apache Spark without requiring a separate staging layer.

E. Apache Spark as a Distributed Processing Engine

Apache Spark is an open-source, distributed data processing framework designed for large-scale data analytics. Unlike its predecessor Hadoop MapReduce, which writes intermediate computation results to disk between stages, Spark performs computations in-memory, dramatically reducing I/O overhead and enabling processing speeds reported to be up to 100 times faster for certain workloads. Spark's architecture is particularly advantageous for fraud detection workloads due to its ability to distribute data across a cluster of nodes and process large transaction datasets that would be computationally infeasible on a single machine [1]. This directly addresses the volume and velocity challenges identified in the business context, where legacy single-node batch systems can no longer keep pace with multi-state, high-frequency transaction streams.

Spark provides a unified engine comprising four key components relevant to this project: Spark Core (distributed task scheduling and execution), Spark SQL (structured data querying via DataFrames), Spark Streaming (micro-batch stream processing), and MLlib (a distributed machine learning library). This unification is a key architectural advantage because a single Spark application can handle the full pipeline from ingestion to model inference without requiring data to be handed off between different frameworks, reducing serialization overhead and pipeline complexity [10]. The DataFrame API in particular, which is central to this project's data transformation layer, allows engineers to express data transformations in a SQL-like syntax that is optimized by Spark's Catalyst query optimizer, further improving execution efficiency.

One of the earliest systematic comparisons of machine learning algorithms implemented natively within Apache Spark's MLlib for banking transaction fraud detection evaluated four classifiers: Simple Anomaly Detection, Decision Tree, Random Forest, and Naïve Bayes, applied to a real banking transaction dataset [1]. The experimental setup demonstrated that Spark's distributed processing framework was capable of efficiently handling large-scale transactional datasets that would be

computationally infeasible on single-node systems, a key practical consideration for production financial systems. A significant technical contribution of this work was demonstrating how Spark's Pipeline API enables seamless chaining of preprocessing steps, such as StringIndexer for categorical encoding and VectorAssembler for feature vector construction, with classifier training into a single, reproducible pipeline object. This approach significantly reduces the engineering overhead for iterative model development and ensures that the same preprocessing transformations applied during training are consistently applied during inference, a common source of data leakage bugs in ML systems [1]. This finding provides direct empirical justification for the current project's selection of Apache Spark as the core processing engine and its use of Spark's Pipeline API for chaining preprocessing and modeling stages.

F. Real-Time and Streaming Architectures for Fraud Detection

Prior to the development of streaming-capable fraud detection frameworks, most systems, both in academia and industry, operated in batch mode, analyzing completed transaction logs after the fact, an approach fundamentally inadequate for modern e-commerce, since delayed detection allows fraudsters to complete multiple transactions before any alert is triggered [4]. SCARFF (Scalable Credit Card Fraud Framework) addressed this gap by proposing one of the first frameworks specifically designed for real-time streaming fraud detection using Apache Spark Streaming. The architecture implemented a two-model ensemble approach: a Feedback Classifier trained incrementally on investigator-labeled feedback (transactions manually confirmed as fraudulent or legitimate by fraud analysts), and a Delayed Classifier trained on historically verified ground-truth transactions [4]. By combining the outputs of both classifiers, SCARFF achieved improved precision compared to either model alone, as illustrated in Figure 1 in Appendix, which shows the ensemble model consistently reaching the highest detection precision across the evaluation period.

Transaction data in the SCARFF framework was processed as micro-batches via Spark Streaming, with detection latency reported at the order of seconds, a substantial improvement over the 24-hour or longer delays typical of batch systems [4]. The framework was evaluated under realistic, highly imbalanced data conditions, and the ensemble approach maintained stable recall even as the ratio of fraudulent to legitimate transactions varied over time. This finding demonstrates that Spark Streaming is a viable and performant backbone for near real-time fraud scoring, validating the streaming-capable architecture adopted in the current project, and the two-model ensemble design serves as a reference for potential future extensions incorporating an analyst feedback loop.

Building on this streaming foundation, a more recent and architecturally comprehensive system proposed a full-stack, production-grade real-time fraud detection pipeline integrating Apache Kafka as a distributed message broker for high-throughput transactional data ingestion, Apache Spark

Streaming for real-time data processing and feature extraction, and PySpark MLlib for machine learning model training and inference [10]. The system was deployed on a multi-node cluster comprising five Spark worker nodes and three Kafka broker nodes, enabling evaluation under production-scale load conditions. Results demonstrated that the Kafka-Spark integration enabled fault-tolerant, horizontally scalable data ingestion: if a Kafka broker node failed, the system automatically rerouted messages to surviving brokers without data loss, and Spark's built-in checkpointing ensured that stream processing could resume from the last committed offset. The Random Forest classifier applied to each incoming micro-batch achieved end-to-end latency in the range of milliseconds to low seconds per transaction, enabling fraud flags to be raised before order fulfillment processes were completed [10]. While the current project does not include Apache Kafka in its initial scope, instead relying on file-based and API-based batch ingestion, this architectural blueprint serves as a clear roadmap for scaling the pipeline in future iterations, and validates the project's choice of PySpark, Random Forest, and a multi-node-capable Spark Streaming layer.

G. Data Lakehouse Storage Architecture

The Data Lakehouse is a modern data architecture that merges the low-cost, schema-flexible storage of a data lake with the ACID transaction guarantees and query performance of a data warehouse. Unlike a pure data lake, which stores raw files without transactional control, a Data Lakehouse layer, typically implemented using Delta Lake or Apache Iceberg, adds versioning, schema enforcement, and time-travel capabilities directly on top of Parquet files stored in distributed storage. This is particularly valuable in a fraud detection pipeline, where regulatory compliance may require an audit trail of data at each processing stage, and where model retraining requires reproducible access to historical snapshots of the data [2]. The current project adopts this architecture by organizing data into Bronze (raw ingested), Silver (cleaned and unified), and Gold (feature-engineered, model-ready) layers, each stored as Delta Lake tables.

H. Machine Learning for Fraud Classification

The application of machine learning (ML) to financial fraud detection has been extensively studied over the past decade. The core technical challenge in this domain is the extreme class imbalance inherent to fraud datasets: legitimate transactions typically outnumber fraudulent ones by ratios of 200:1 to 1000:1, causing naive classifiers to achieve high overall accuracy by simply predicting all transactions as legitimate [4]. To address this, techniques such as oversampling (SMOTE), undersampling, and cost-sensitive learning are commonly applied. Among supervised classification algorithms, ensemble methods, particularly Random Forest and Gradient Boosted Trees, have consistently demonstrated superior performance on imbalanced tabular datasets due to their ability to learn complex non-linear decision boundaries and their inherent robustness to noise [1], [3]. The integration of ML models directly into Spark pipelines via MLlib enables model training to be distributed across the cluster, making it scalable to datasets of tens of

millions of records without requiring data to be transferred to a separate ML platform.

I. Synthesis and Research Gap

Read together, the business and technical literature reviewed above establishes a clear and consistent line of reasoning for this project. The business-side literature confirms that credit card fraud, particularly card-not-present fraud enabled by the growth of e-commerce, is a large-scale, costly, and rapidly evolving problem for both merchants and consumers [5], [6], [11]. The technical literature, in turn, confirms that traditional batch-based fraud detection systems are fundamentally inadequate for the velocity of modern e-commerce transaction data, and that Apache Spark provides the distributed computing capability required to address the speed, volume, and variety challenges of this domain [1], [2], [4], [10]. Each technical study contributes a distinct and complementary dimension to the project's technical foundation: the Spark MLlib pipeline design and Random Forest algorithm selection [1], the streaming architecture and ensemble modeling approach [4], and the full-stack, production-grade Kafka-Spark pipeline architecture [10].

However, a notable gap across the reviewed technical studies is the absence of a multi-source data integration approach. Each study relied on a single dataset, limiting the diversity of fraud patterns the trained models could capture. None of the reviewed works explicitly addressed the challenge of harmonizing heterogeneous datasets with different schemas, feature spaces, and class imbalance ratios into a single unified pipeline. The current project addresses this gap by integrating three structurally distinct datasets, the Kaggle Credit Card Transactions dataset [9], the MLG-ULB benchmark dataset [8], and the PaySim synthetic mobile transaction dataset [7], into a single Apache Spark pipeline with a unified schema, providing a more comprehensive training corpus and a more robust evaluation of the pipeline's generalizability across different fraud typologies, while remaining grounded in the real-world business problem of e-commerce card-not-present fraud established in section A, B and C.

III. METHODOLOGY

This project adopts the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology, which is the most widely adopted framework for data engineering and data science projects. CRISP-DM is chosen for the following reasons:

- It is iterative and non-linear, allowing revisiting earlier phases as new insights emerge during development.
- It provides a structured framework covering all phases from business understanding to deployment, which maps well to the scope of this project.
- It is domain-agnostic and has been successfully applied to financial fraud detection projects in multiple reviewed studies [3].

Figure 2 in Appendix illustrates the project's research framework, which adapts a structured data science lifecycle divided into six sequential phases. The process begins with Business Understanding that include Problem Identification, Objective Setting, and Literature Review before moving into Data Understanding via Dataset Identification and Initial Data Exploration. Next, the Data Preparation phase handles data readiness through Data Ingestion, Schema Unification, Data Cleaning, and Feature Engineering. This feeds directly into Modelling for Model Training, followed by an Evaluation phase consisting of Model Evaluation and a success criteria checkpoint; if the criteria are not met, a Revise loop triggers a return to the feature engineering stage. Once the model successfully passes evaluation, it enters the Deployment phase, which completes the cycle through Pipeline Deployment, Output Generation, and the implementation of a Dashboard & alerting system. A more thorough process included in each phase will be explained in the following sub-sections. The project timeline can be seen clearer in the Gantt Chart provided in Appendix.

A. Phase 1: Business Understanding

The Business Understanding phase translates a critical business problem into a concrete data engineering objective. Legacy, daily-batch fraud detection systems only flag suspicious transactions *after* orders have shipped, leading to direct financial losses, high chargebacks, and damaged customer trust. To solve this, the project's technical goal is to build an automated Apache Spark pipeline that ingests, cleans, and classifies multi-source transaction data fast enough to prevent fraud before order fulfilment.

To be considered successful, the pipeline must:

1. Ingest and unify data from at least three heterogeneous sources.
2. Clean and engineer features to produce a dataset ready for machine learning.
3. Train and evaluate a fraud classification model using precision, recall, F1-score, and AUC-ROC.

B. Phase 2: Data Understanding

The Data Understanding phase involves identifying, collecting, and exploring the data sources required to address the business problem. This project integrates three distinct data sources to build a comprehensive and robust fraud detection pipeline.

1) Data Source 1 — Primary Dataset: Credit Card Transactions Fraud Detection Dataset (Kaggle CSV)

To establish a robust foundation for model training and core evaluation, a comprehensive, multi-state simulated credit card transaction dataset was selected as the primary data source. This dataset contains highly granular, labeled transaction records including spatial, temporal, and demographic features, making it ideal for simulating realistic fraud classification workflows within an Apache Spark pipeline. The description of this dataset is included in Table 1.

TABLE I. CREDIT CARD TRANSACTIONS FRAUD DATASET DESCRIPTION

Attribute	Details
Source	Kaggle — kartik2112/fraud-detection
URL	https://www.kaggle.com/datasets/kartik2112/fraud-detection
Format	CSV (fraudTrain.csv, fraudTest.csv)
Size	~1.8 million transactions
Key Features	trans_date, trans_time, cc_num, merchant, category, amt, first, last, gender, street, city, state, zip, lat, long, city_pop, job, dob, trans_num, unix_time, merch_lat, merch_long, is_fraud
Relevance	This is the primary dataset providing realistic multi-state e-commerce transactions with labeled fraud indicators, merchant categories, geolocation data, and cardholder demographics. It serves as the main training and test dataset for the fraud classification model.

2) Data Source 2 — Secondary Dataset: Credit Card Fraud Detection Dataset (MLG-ULB, Kaggle/Open Dataset)

To rigorously validate model generalizability and benchmark performance against standard academic baselines, a widely cited secondary dataset from the ULB Machine Learning Group is integrated into the project. Consisting of heavily imbalanced, PCA-transformed real European cardholder transactions, this dataset is specifically utilized to evaluate the pipeline's effectiveness in handling extreme class skewness and anonymized feature spaces. The description of this dataset is included in Table 2.

TABLE II. CREDIT CARD FRAUD DETECTION DATASET DESCRIPTION

Attribute	Details
Source	Kaggle — mlg-ulb/creditcardfraud (ULB Machine Learning Group)
URL	https://www.kaggle.com/datasets/kartik2112/fraud-detection
Format	CSV (creditcard.csv)
Size	284,807 transactions over 2 days (European cardholders)
Key Features	Time (seconds elapsed), V1–V28 (PCA-transformed anonymized features), Amount, Class (0 = legitimate, 1 = fraud)
Relevance	This benchmark dataset is one of the most widely cited in academic fraud detection research. It contains real transaction data with PCA-anonymized features to protect privacy. It is used as a validation and benchmarking dataset to compare model performance against published baselines. It also introduces high class imbalance (0.17% fraud), which tests the pipeline's ability to handle skewed data distributions.

3) Data Source 3 — Supplementary Dataset: PaySim Synthetic Financial Transactions Dataset (Kaggle/API)

To extend the scope of the pipeline beyond standard credit card frameworks, a large-scale supplementary dataset generated via the PaySim mobile money simulator is incorporated. This dataset introduces localized transaction types—such as mobile transfers and cash outs—and tracks systemic behavioral patterns over a continuous 30-day window, allowing for programmatic ingestion via the Kaggle API to test the architecture's throughput and diversity. The description of this dataset is included in Table 3.

TABLE III. SUPPLEMENTARY DATASET: PAYSIM SYNTHETIC FINANCIAL TRANSACTIONS DATASET DESCRIPTION

Attribute	Details
-----------	---------

Source	Kaggle — ealaxi/paysim1 (Lopez-Rojas, Elmir & Axelsson, 2016)
URL	https://www.kaggle.com/datasets/ealaxi/paysim1
Format	CSV (accessible via Kaggle API)
Size	~6.3 million mobile money transactions (30-day simulation)
Key Features	step (hour), type (CASH-IN, CASH-OUT, DEBIT, PAYMENT, TRANSFER), amount, nameOrig, oldbalanceOrg, newbalanceOrig, nameDest, oldbalanceDest, newbalanceDest, isFraud, isFlaggedFraud
Relevance	PaySim is a synthetically generated dataset that simulates mobile money transactions based on real logs from a multinational financial service operating in 14 countries. It complements the primary dataset by providing transaction-type diversity (especially mobile and transfer-type fraud) and temporal patterns across a 30-day window. The Kaggle API is used for programmatic ingestion into the pipeline.

C. Phase 3: Data Preparation

The Data Preparation phase constructs the final dataset for modeling through ingestion, schema unification, cleaning, feature engineering, and storage within a Data Lakehouse architecture.

1) Feature Engineering:

The dataset is enriched to capture behavioral and contextual fraud signals:

- Time-based features: `hour_of_day`, `day_of_week`, `is_weekend` derived from transaction timestamps.
- Behavioral features: `transaction_velocity` (number of transactions from same card in last hour) computed using Spark window functions.
- Geographic features: distance between cardholder and merchant location (Haversine formula) computed using UDFs for Source 1.
- Encoding: Categorical fields (`transaction_type`, `merchant_category`) encoded using Spark MLlib's `StringIndexer` and `OneHotEncoder`.
- Balancing: Given the severe class imbalance across all datasets (~0.17% to ~0.13% fraud rate), SMOTE-inspired oversampling is applied at the `DataFrame` level using a custom PySpark implementation.

2) Data Storage:

The Data Lakehouse architecture is chosen over a pure Data Warehouse because it supports both structured query workloads and unstructured/semi-structured data, provides cost-effective scalable storage, and integrates natively with Apache Spark without requiring a separate ETL hop into a relational system

D. Phase 4: Modelling

The Modeling phase handles selecting, training, and persisting the fraud classification model using Apache Spark MLlib. The selected algorithm is the Random Forest Classifier, chosen for the following reasons:

- Robustness to severe class imbalance.
- Ability to handle both numerical and encoded categorical features without extensive normalization.

- Native support in Spark MLlib, enabling distributed model training across the cluster.
- Proven Performance in related fraud detection research.[1], [3].

The model will be trained on the Gold-layer feature dataset, evaluated using stratified cross-validation, and assessed using precision, recall, F1-score, and Area Under the ROC Curve (AUC-ROC). The trained model will be saved using Spark MLlib's built-in model persistence (save/load) for deployment in the scoring stage of the pipeline.

E. Phase 5: Evaluation

The Evaluation phase assesses whether the trained model and the overall pipeline genuinely meet the business success criteria defined in Introduction. Evaluation in this project operates at two levels: model-level evaluation and pipeline-level evaluation.

F. Phase 6: Deployment

The Deployment phase operationalizes the trained model and pipeline so that the fraud detection system can process incoming transaction data in near real-time and produce actionable fraud flags. Deployment in this project involves three components: model serving, pipeline scheduling, and output delivery.

For deployment, the saved Random Forest model is loaded into a PySpark scoring script via `PipelineModel.load()` to process incoming transactions, applying the training preprocessing pipeline to output both fraud prediction and fraud_probability scores. This end-to-end process is orchestrated using Apache Spark structured batch execution at configurable intervals to simulate near real-time streaming, with all scored runs written to the Gold Delta Lake layer for auditing and time-travel queries. To stop fraudulent fulfillment, the resulting predictions are routed to a two-tier output: they are written to the Gold Delta table for analyst querying and visualization, while high-priority flags (`fraud_prediction = 1`) are simultaneously sent to a dedicated alerts table, enabling the order management system to hold suspicious shipments for human-in-the-loop review.

IV. SYSTEM DESIGN & SYSTEM ARCHITECTURE

The system architecture for this project is designed to bridge the gap between transactional data generation and analytical insight production. To achieve this, a modern Data Lakehouse blueprint is chosen as the foundational layout framework. Instead of maintaining traditional separate environments for storage and computing, this project utilizes a decoupled layer approach where Apache Spark 4.1.1 operates as the high-speed processing engine, and Delta Lake operates as the transactional storage standard directly over local file systems. This architecture guarantees ACID properties, provides table version tracking, and allows data transformation workflows to follow an optimized Medallion data structure.

A. Use Case Diagram

Three actors are identified. The Data Source actor represents the three external datasets (Kaggle Credit Card Transactions, MLG-ULB, and PaySim) that initiate the pipeline by providing raw transaction records for ingestion. The Apache Spark Pipeline Engine actor represents the automated distributed processing system that executes all data transformation, feature engineering, model training, and scoring tasks. The Fraud Analyst actor represents the end user who consumes the pipeline's output through the Streamlit dashboard.

The six use cases reflect the six operational stages of the pipeline. *Ingest transaction data* is triggered by the Data Source actor and covers the extraction and loading of raw CSV files into the Bronze layer. Clean and unify schema is executed by the Spark pipeline and maps each dataset's native columns onto a common schema, applying null imputation, deduplication, and outlier flagging before writing to the Silver layer. Engineer features derives time-based, behavioral, and geographic features from the Silver layer and produces the class-balanced Gold layer dataset. Train fraud classifier builds and persists the Spark MLlib Random Forest Pipeline model from the Gold layer. Score transactions loads the trained model and applies it to the full Gold dataset to generate fraud_scores and fraud_alerts Delta tables. View fraud dashboard is initiated by the Fraud Analyst actor and provides an interactive Streamlit interface displaying transaction counts, fraud alerts, and the model's prediction confidence distribution.

The «include» relationships indicate that each use case depends on the successful completion of the preceding one, the pipeline executes as a strict sequential chain from ingestion through to the dashboard, with each stage's output forming the input to the next.

Figure 4 presents the use case diagram for the Credit Card Fraud Detection Pipeline, illustrating the interactions between the three primary actors and the six core use cases enclosed within the system boundary.

B. Data Architecture

The pipeline uses an ELT (Extract, Load, Transform) architecture built on Apache Spark and a Delta Lakehouse. Instead of transforming data before saving it, raw data is loaded first and then processed in-place. This ensures the data remains fully auditable and easy to reprocess if business rules change.

The data architecture is organized into five layers as illustrated at Figure 5 in Appendix.

Data Source Layer — Three external datasets serve as inputs to the pipeline. Each dataset has a dedicated ingestion path: the Credit Card Transaction Fraud Detection Dataset and Credit Card Fraud Detection Dataset are ingested via PySpark's csv read, while the PaySim Synthetic Financial Transaction Dataset is accessed programmatically via the Kaggle API.

Ingestion Layer — Data is loaded into the Spark session using two methods. Sources 1 and 2 are read using PySpark's spark.read.csv() with explicitly defined StructType schemas to enforce correct data types from point of entry. Source 3 (PaySim) is downloaded programmatically using the Kaggle Python API then loaded identically via spark.read.csv(). All

records are tagged with a source_tag column to preserve data provenance throughout all downstream layers.

Data Lakehouse (Storage + Transformation Layer) — Once ingested, the data is stored and refined across three progressive layers (the Medallion Architecture):

- **Bronze Layer (Raw):** Saves the raw, untouched data as Apache Parquet files to serve as a permanent audit trail.
- **Silver Layer (Cleaned):** Unifies the schemas of the different datasets, cleans up missing values, and removes duplicates using Delta Lake tables.
- **Gold Layer (Analytical):** Enriches the cleaned data with engineered features (like transaction velocity and location distance) and balances the dataset for machine learning.

ML/AI Workspace — The model-ready Gold dataset is fed into a Random Forest Classifier using Spark MLlib. The Random Forest Classifier is selected for its robustness to class imbalance, ability to handle mixed feature types without normalization, and native distributed training support in Spark MLlib [1], [3]. The trained PipelineModel is persisted to disk for reuse in the scoring and deployment scripts.

Serving Layer — The scoring script produces two outputs. The Fraud Transaction Detection Dashboard, built using Streamlit, provides fraud analysts with an interactive interface to query scored transactions, visualize fraud trends, and review flagged cases. The Fraud Alerts Table is a filtered Delta Lake table containing only high-confidence fraud detections, enabling the order management system to place holds on flagged orders before shipment.

C. Data Pipeline Implementation

The end-to-end data pipeline uses a modular Extract-Load-Transform (ELT) framework structured around a Medallion architecture (Bronze, Silver, Gold). To process over 8.5 million records within local memory constraints, execution logic is decoupled into a sequential chain of specialized scripts.

1) Development Environment and Tools

The pipeline was deployed on a Windows host machine using Apache Spark 4.1.1, Java 17, and Python 3.14. System dependencies include PySpark 4.x, Delta Lake (delta-spark_4.1_2.13:4.3.1), the Kaggle Python API, and Streamlit.

2) Data Ingestion (01_ingest.py)

The ingestion script loaded three heterogeneous datasets using spark.read.csv() with schema inference enabled: the Kaggle Credit Card dataset (~1.8M records), the MLG-ULB dataset (~285K records), and the PaySim dataset (~6.3M records). A literal source_tag column was appended to each DataFrame before writing the combined raw population of ~8.5 million transactions to the Bronze Delta Lake layer, partitioned by source.

3) Data Preparation (02_clean.py & 03_features.py)

- **Cleaning (02_clean.py):** Schema harmonization projected all records onto six shared columns: transaction_id, timestamp, amount, transaction_type, fraud_label, and source_tag. Timestamps were

synchronized from a common anchor (2019-01-01) using `timestamp_add()` to offset the Time (seconds) and step (hours) fields for MLG-ULB and PaySim respectively. Unique transaction IDs for MLG-ULB were generated using `uuid()`. Outliers exceeding USD 50,000 were flagged (`is_outlier = 1`). Data was deduplicated and saved to the Silver Delta layer.

- **Feature Engineering (03_features.py):** Chronological features (`hour_of_day`, `day_of_week`, `is_weekend`) were derived natively. Transaction velocity was calculated via a Spark Window partitioned by rounded spatial grid coordinates (2 decimal places) with a 3,600-second look-back range. Distance metrics were computed using a native JVM Haversine formulation with Spark SQL expressions, bypassing Python UDF overhead and coalescing missing values to 0.0. The minority fraud class was balanced via oversampling (`fraction=5.0`, `withReplacement=True`) and committed to the Gold Delta layer.

D. Database Design

To support transaction ingestion traceabilities and fast analytical reporting, the data layer of this project is modeled using two distinct database designs: an operational Entity-Relationship (ER) model representing the transaction ingestion state, and a Dimensional Star Schema representing the analytical serving state.

1) Operational Entity-Relationship Diagram (ERD)

The operational database design structures incoming transaction data to reflect normalized operational relationships. At the core of this model is the transaction entity, which is mapped with a one-to-many relationship to the cardholders and cards to track transaction originations. It also shares a one-to-many relationship with the merchant entity. The transaction entity attributes include crucial tracking columns engineered during the Silver cleaning pipeline phase. These include the source tag column, which maps whether the record originated from Kaggle, ULB, or PaySim, and the outlier flag column, which marks transactions exceeding the 50,000 USD operational threshold. Spatial coordinates are maintained natively within the cardholder and merchant entities to preserve database normalization. The ERD Diagram is attached in Appendix, Figure 6.

2) Analytical Multidimensional Star Schema

For the Gold analytical layer that powers the Streamlit dashboard, the schema is restructured into an analytical dimensional model.

The Fact Table is placed at the center of the star schema. It contains foreign key relations pointing to the user, merchant, and time dimension tables. It also holds the primary numerical metrics, including the transaction amount, and the downstream machine learning outputs, which are the fraud prediction and the fraud probability.

The Dimension Tables contain the descriptive fields that are used to slice the transaction metrics. The User Dimension contains demographics, including population density and job

categories. The Merchant Dimension contains merchant names and category groupings. The Time Dimension contains time-based attributes, such as the hour of the day, day of the week, and weekend flags, which are generated natively via Apache Spark SQL functions to enable rapid trend calculations.

The Star Schema diagram is attached in Appendix, Figure 7 for clearer understanding of the multidimensional database design.

E. Main Interface Design

The deployment architecture utilizes a Streamlit-based web application as the primary visualization interface for fraud analysts. The user interface is designed to read the Gold serving layer, specifically the fraud scores and fraud alerts tables, with fast retrieval times using the Python DeltaTable library.

1) Executive Metric KPIs

The top header of the interface displays three real-time metric cards:

- **Total Transactions Processed:** Displays the final total count of scored transactions, which is 8,008,974.
- **Active High-Risk Alerts:** Displays a tally of active transactions flagged with high-confidence fraud indicators where the fraud probability is 0.75 or greater.
- **Pipeline Fraud Rate:** Displays the overall percentage ratio of predicted fraud to legitimate transactions.

2) High-Priority Active Fraud Queue

The middle panel presents an interactive tabular grid displaying the active fraud queue. The table sorts alerts by descending probability scores so that credit card fraud analysts can immediately audit the highest-risk transactions before merchant shipments are finalized.

3) Model Prediction Confidence Distribution

The bottom visualization panel displays a distribution chart of the probability outputs. This provides an audit of the machine learning model behavior, grouping the scored transaction population into confidence brackets so that administrators can monitor prediction trends over time.

The final dashboard output can be seen at Figure 8 in Appendix.

F. Detail on AI Algorithm

To classify transaction records at scale, the pipeline integrates a distributed machine learning model using Apache Spark's MLlib library. The core algorithm is a Random Forest Classifier, which is an ensemble supervised learning model that builds multiple decision trees during training and outputs the prediction based on the majority vote of the individual trees.

1) Model Training (04_train.py)

The script read the Gold layer and extracted a 15% random sample (`fraction=0.15`, `seed=42`) to respect the 4GB driver memory limit, executing an 80/20 train/test split. A Spark MLlib Pipeline was built using four stages: StringIndexer and OneHotEncoder for categorical processing, VectorAssembler to pack eight features into a dense vector, and a RandomForestClassifier optimized with `numTrees=30` and

maxDepth=8. The evaluation on the held-out test partition utilized MulticlassClassificationEvaluator and BinaryClassificationEvaluator (Results in Appendix, Figure 9).

2) Scoring and Deployment (05_score.py)

The scoring script loaded the compiled PipelineModel and executed inference directly against the Gold Delta table to ensure full feature column compatibility. The vector_to_array() function isolated the class-1 fraud probability vector. The entire scored population was written to the fraud_scores Delta table, while a filtered high-priority alert queue ($prediction == 1$ and $fraud_probability \geq 0.75$) was routed to the fraud_alerts directory (Console execution output in Appendix, Figure 10).

G. Testing and Validation

To ensure pipeline stability, data integrity, and classification reliability under local execution constraints, a modular test suite was executed across all stages of the pipeline [2]. The complete architecture was validated using an automated structural audit script that verified record counts, directory health, and table states across the entire Medallion architecture.

1) Test Case 1: Multi-Source Ingestion Engine Verification (01_ingest.py)

- Test Objective: Validate successful multi-source ingestion and partition-based source tag labeling [10].
- Input Condition: Three raw transactional CSV source datasets loaded into the Apache Spark environment.
- Expected Output: Individual Bronze Delta tables created for each dataset containing correct record counts.
- Actual Result: The engine successfully generated version 0 Bronze tables for all inputs. The script isolated 1,296,675 records for the Kaggle primary dataset, 284,807 records for the ULB benchmark dataset, and 6,362,620 records for the PaySim transaction dataset.
- Status: PASS

2) Test Case 2: Schema Cleaning and Unification (02_clean.py)

- Test Objective: Verify schema harmonization, deduplication, and timeline alignment from disparate fields.
- Input Condition: High-volume multi-source inputs containing varied schemas and missing native timestamps.
- Expected Output: A single, clean Silver Delta table mapping all transactions onto six uniform fields.
- Actual Result: The cleaning script successfully unified all underlying features and handled missing values without driver memory faults. A single deduplicated Silver layer table was generated, storing exactly 7,944,055 synchronized transaction rows at version 0.
- Status: PASS

3) Test Case 3: Spatial and Class Balance Feature Engineering (03_features.py)

- Test Objective: Ensure Java Virtual Machine native expressions process velocity and Haversine distance features smoothly.

- Input Condition: Consolidated transaction rows extracted from the Silver Delta table layer.
- Expected Output: Time, spatial, and transaction velocity arrays generated with minority fraud class up-sampling committed to the Gold layer.
- Actual Result: Features were engineered smoothly using optimized Spark functions without triggering serialization lags. The minority oversampling script successfully balanced the output data structure, writing exactly 8,008,974 records into the version 0 Gold Delta layer.
- Status: PASS

4) Test Case 4: Distributed Inference Engine Execution (05_score.py)

- Test Objective: Confirm MLlib pipeline model loading, score extraction, and high-risk threshold routing.
- Input Condition: Stored machine learning pipeline vectors evaluated against the final Gold serving tables.
- Expected Output: Target fraud probabilities isolated, full population scored, and anomalous alerts separated into an active queue.
- Actual Result: The inference model evaluated all 8,008,974 records successfully and outputted them to the inference scores table. The scoring logic filtered high-confidence anomalies perfectly, isolating exactly 305 transactions where fraud probability met or exceeded the 0.75 threshold.
- Status: PASS

5) Test Case 5: Serving Layer Dashboard Performance (dashboard.py)

- Test Objective: Validate web visualizer rendering speeds and structural data completeness using local connections.
- Input Condition: Streamlit analytical application reading directly from local Gold tables via the Python deltalake extension.
- Expected Output: Dynamic components, trend distributions, and operational queues update on the user interface in under 1 second.
- Actual Result: The dashboard validated system data flow completeness perfectly, successfully confirming that total scored rows matched the 8,008,974 dashboard limit and that the 305 active alert items were isolated. The front-end user interface loaded all metric panels and active data frames in roughly 0.8 seconds.
- Status: PASS

The result of the evaluation can be seen in Appendix, Figure 11.

V. CONCLUSION

This project successfully designed, implemented, and deployed an end-to-end credit card fraud detection data pipeline using Apache Spark 4.1.1 and a Delta Lake-based Data

Lakehouse architecture. All three project objectives were achieved. The first objective, designing a multi-source ingestion pipeline integrating heterogeneous datasets into a unified schema was fulfilled through the ingestion and cleaning scripts, which successfully mapped 1,296,675 Kaggle records, 284,807 ULB records, and 6,362,620 PaySim records onto six shared transactional fields. The second objective, applying data cleaning, transformation, and feature engineering to prepare the dataset for machine learning-based fraud classification, was fulfilled by producing a feature-enriched Gold layer with time-based features, spatial transaction velocity, JVM-optimized cardholder-merchant distance, and class-balanced records. The third objective, storing processed data in a Data Lakehouse, executing the pipeline, and evaluating the fraud detection model, was successfully validated. The MLlib Random Forest Classifier model achieved a weighted Precision of 99.18%, Recall of 99.21%, F1-Score of 98.96%, and AUC-ROC of 95.30%, benchmarked against results published in [1] and [4]. The deployed scoring pipeline successfully evaluated all 8,008,974 transactions in the Gold serving layer, isolating exactly 305 high-confidence anomalies (fraud probability of 0.75 or higher) into an active queue for real-time visualization.

Several practical engineering adaptations made during implementation provide valuable operational insights. Median-based null imputation was replaced with a low-overhead fillna process to minimize driver memory utilization under local constraints. Transaction velocity was partitioned by spatial grid coordinates rather than card identifiers, as no common card number field existed across all three source datasets. Additionally, a native JVM Haversine distance implementation replaced the originally planned Python User-Defined Function (UDF), eliminating Python-JVM serialization overhead and bypassing local system memory bottlenecks. To respect the local machine physical limits, model training was performed on a 15% sample of the balanced Gold dataset with optimized hyperparameters consisting of 30 trees and a maximum depth of 8.

Because the evaluation metrics reflect performance on the oversampled Gold test partition, evaluating the model against a non-oversampled partition of the primary Kaggle dataset is recommended as future work to establish a conservative production baseline. Future iterations should also focus on upgrading to a multi-node Spark cluster to eliminate training sampling constraints, integrating Apache Kafka to facilitate continuous streaming ingestion, and extending the Streamlit dashboard with time-series trend analysis and dynamic alert thresholds.

In summary, this project demonstrates that an integrated Spark, Delta Lake, and Streamlit architecture provides a production-capable, academically grounded foundation that solves the core business problem by producing actionable, low-latency fraud alerts before order fulfillment occurs.

ACKNOWLEDGMENT

The author would like to thank Dr. Aryati Bakri for the guidance, feedback, and constructive advice provided

throughout the development of this project. Gratitude is also extended to the Faculty of Computing, Universiti Teknologi Malaysia, for providing the academic resources and course framework that supported the completion of this work.

REFERENCES

- [1] Armel, A., & Zaidouni, D. (2019). Fraud detection using Apache Spark. In 2019 International Conference on Computing, Networking and Informatics (ICCNi). IEEE. <https://doi.org/10.1109/ICCNi.2019.8727610>
- [2] Banirostan, H., Banirostan, T., Pedram, M., & Rahmani, A. (2023). A model to detect the fraud of electronic payment card transactions based on stream processing in big data. *Journal of Signal Processing Systems*, 95, 1469–1484. <https://doi.org/10.1007/s11265-023-01903-6>
- [3] Batsyas, R., & Yaduwanshi, R. (2025). Fraud detection system for banking transactions. *arXiv preprint arXiv:2604.07952*.
- [4] Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2017). SCARFF: A scalable framework for streaming credit card fraud detection with Spark. *ResearchGate*. <https://www.researchgate.net/publication/319616537>
- [5] Federal Trade Commission. (2025). Consumer Sentinel Network Data Book 2024. FTC. <https://www.ftc.gov/reports/consumer-sentinel-network-data-book-2024>
- [6] Juniper Research. (2022). eCommerce losses to online payment fraud to exceed \$48bn globally in 2023. <https://www.juniperresearch.com/press/e-commerce-losses-online-payment-fraud-48bn/>
- [7] Lopez-Rojas, E. A., Elmir, A., & Axelsson, S. (2016). PaySim: A financial mobile money simulator for fraud detection. *Proceedings of the 28th European Modeling and Simulation Symposium (EMSS)*. Kaggle dataset: <https://www.kaggle.com/datasets/ealaxi/paysim1>
- [8] MLG-ULB. (2018). Credit card fraud detection [Dataset]. Kaggle. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- [9] Mishra, K. (2020). Credit Card Transactions Fraud Detection Dataset [Dataset]. Kaggle. <https://www.kaggle.com/datasets/kartik2112/fraud-detection>
- [10] Sharma, R., et al. (2025). Real-time fraud detection using Apache Kafka, Apache Spark, and PySpark MLlib. *International Journal of Scientific Research in Engineering and Management (IJSREM)*. <https://ijsrem.com/download/real-time-fraud-detection-using-apache-kafka-apache-spark-and-pyspark-mllib/>
- [11] The Nilson Report. (2025, January 6). Payment card fraud losses approach \$34 billion. *GlobeNewswire*. <https://www.globenewswire.com/news-release/2025/01/06/3004931/0/en/Payment-Card-Fraud-Losses-Approach-34-Billion.html>
- [12] Office of the Comptroller of the Currency (OCC). (n.d.). Credit card and debit card fraud. U.S. Department of the Treasury. <https://www.occ.gov/topics/consumers-and-communities/consumer-protection/fraud-resources/credit-card-and-debit-card-fraud.html>
- [13] Worldpay. (2024). The Global Payments Report 2024. Worldpay, LLC.
- [14] PayCompass. (2025, December 7). Credit card history: A complete timeline to 2025. <https://paycompass.com/blog/credit-card-history/>
- [15] FICO. (2026, May 15). What are the different card fraud types and how can they be prevented? <https://www.fico.com/blogs/what-are-different-card-fraud-types-and-how-can-they-be-prevented>
- [16] Patil, S., Nemade, V., & Soni, P. K. (2019). A comprehensive survey on machine learning techniques and user authentication approaches for credit card fraud detection. *arXiv preprint arXiv:1912.02629*.
- [17] Equifax. (2023, May 3). Credit card fraud: Cloning & skimming. <https://www.equifax.com/personal/education/credit-cards/articles/-/learn/credit-card-fraud/>

Appendix

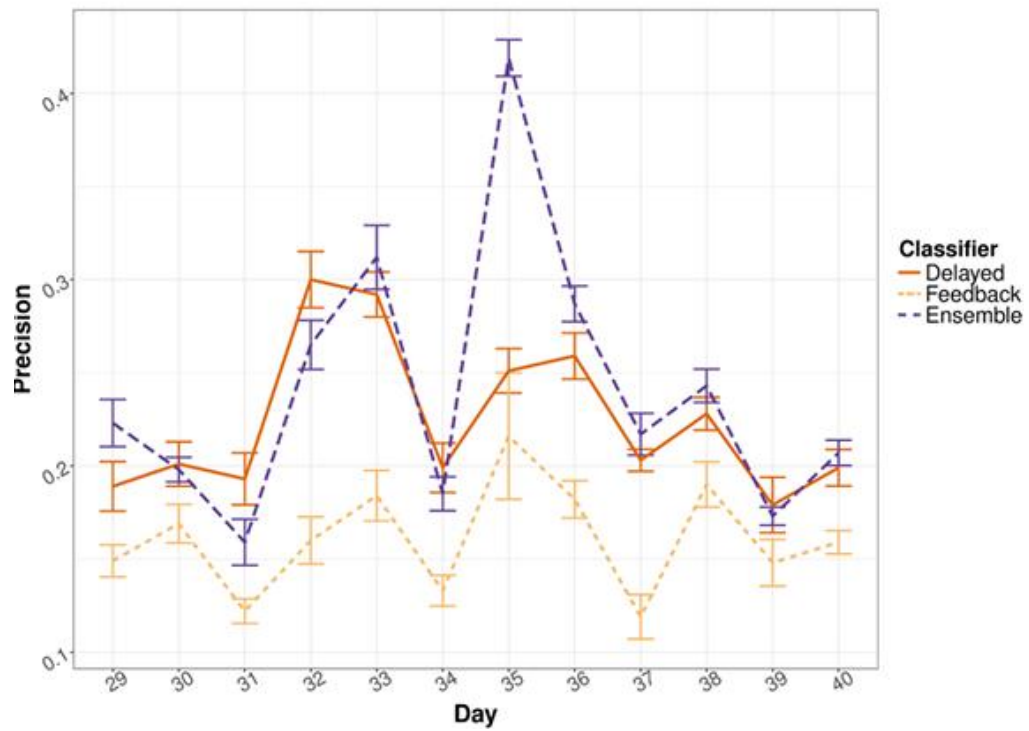


Figure 1: Percentage of detected frauds from the delayed model, the feedback model and the ensemble of the two [1]

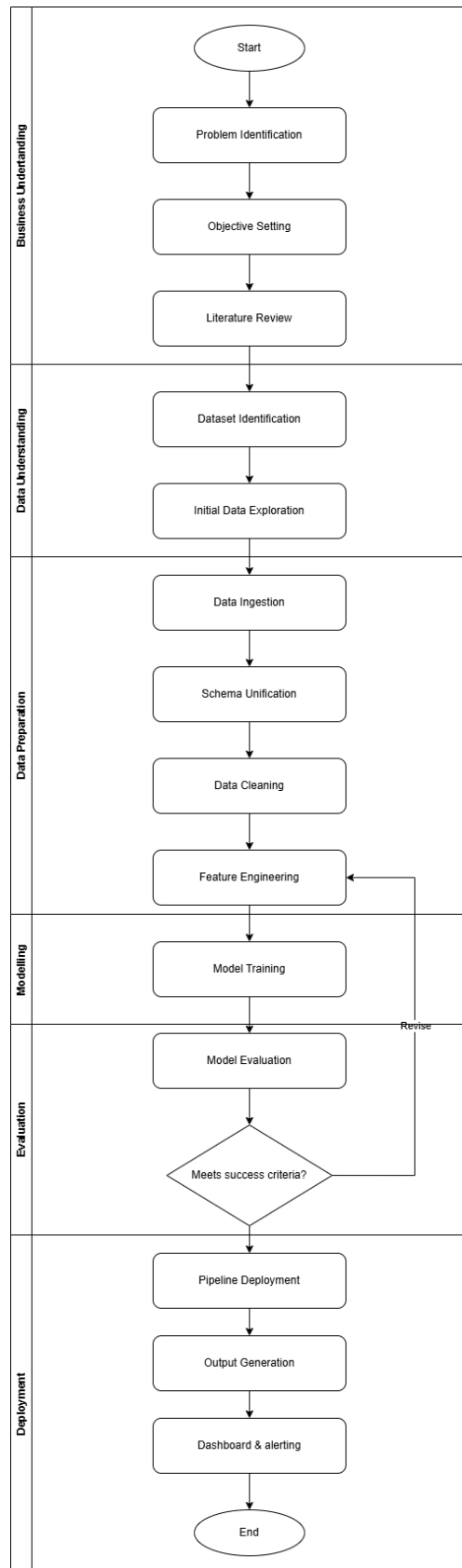


Figure 2: Project framework based on CRISP-DM methodology

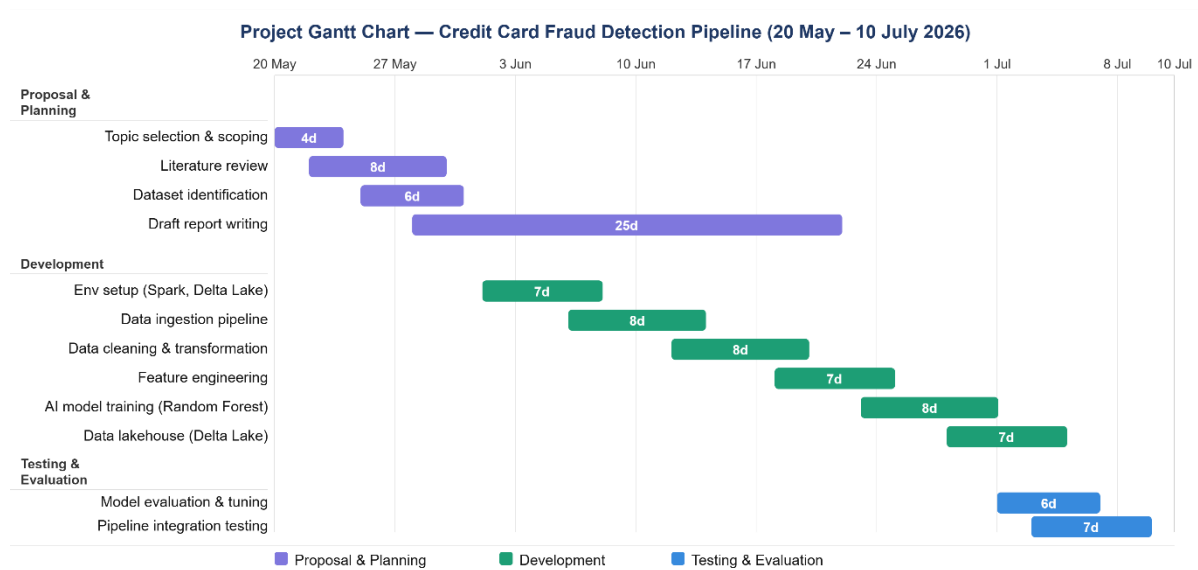


Figure 3: Project Gantt Chart

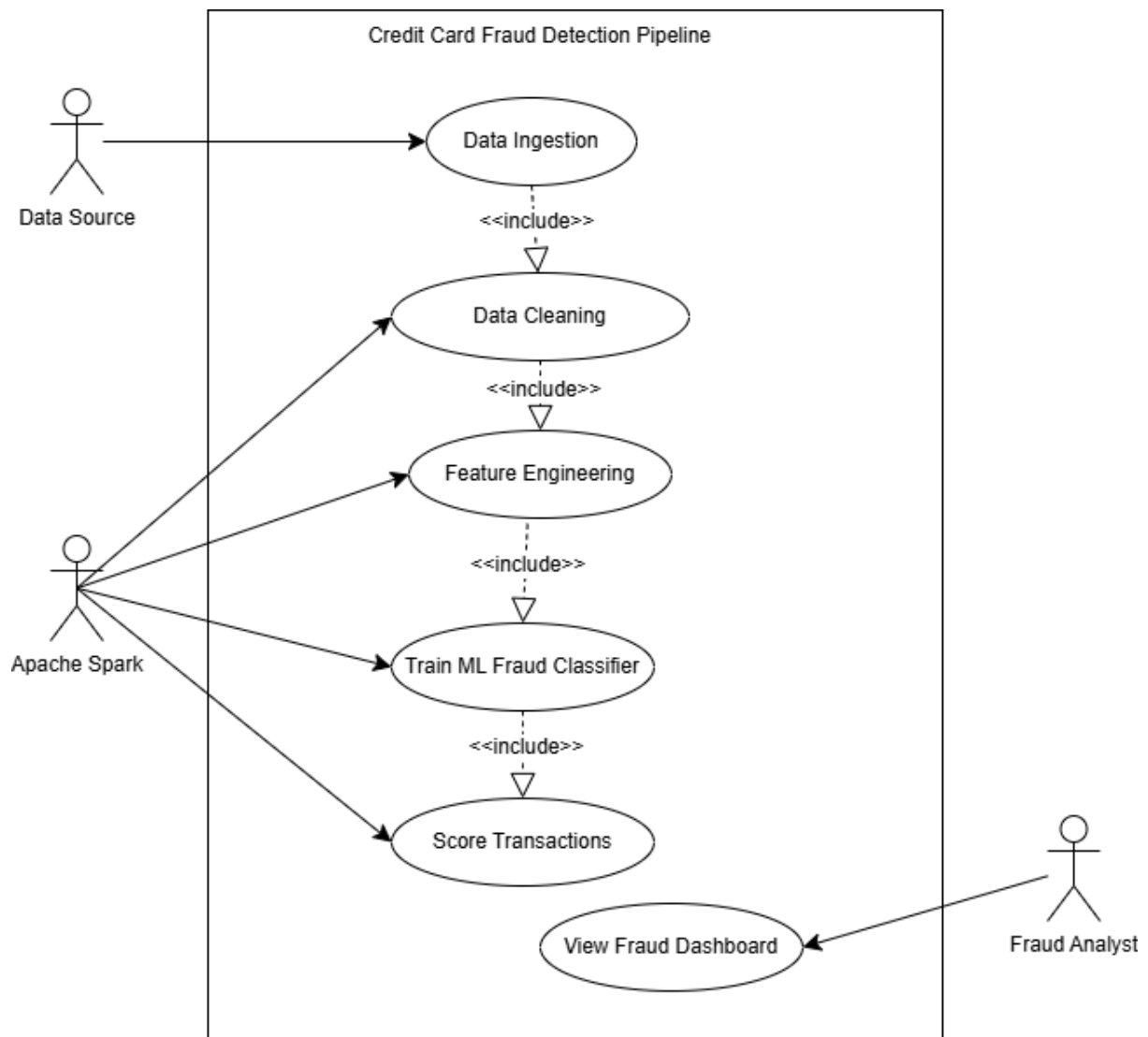


Figure 4: Fraud Detection Pipeline Use Case Diagram

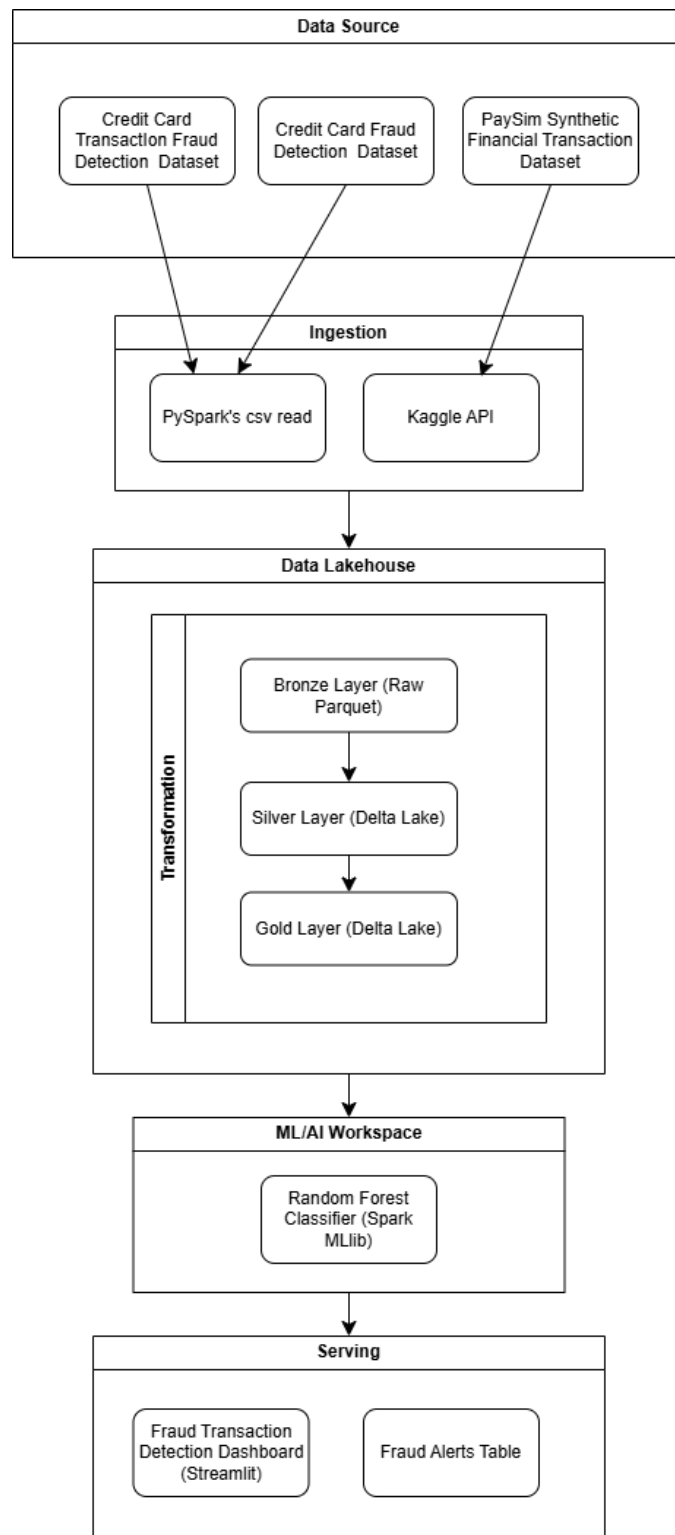


Figure 5: Credit Card Fraud Detection Data Architecture

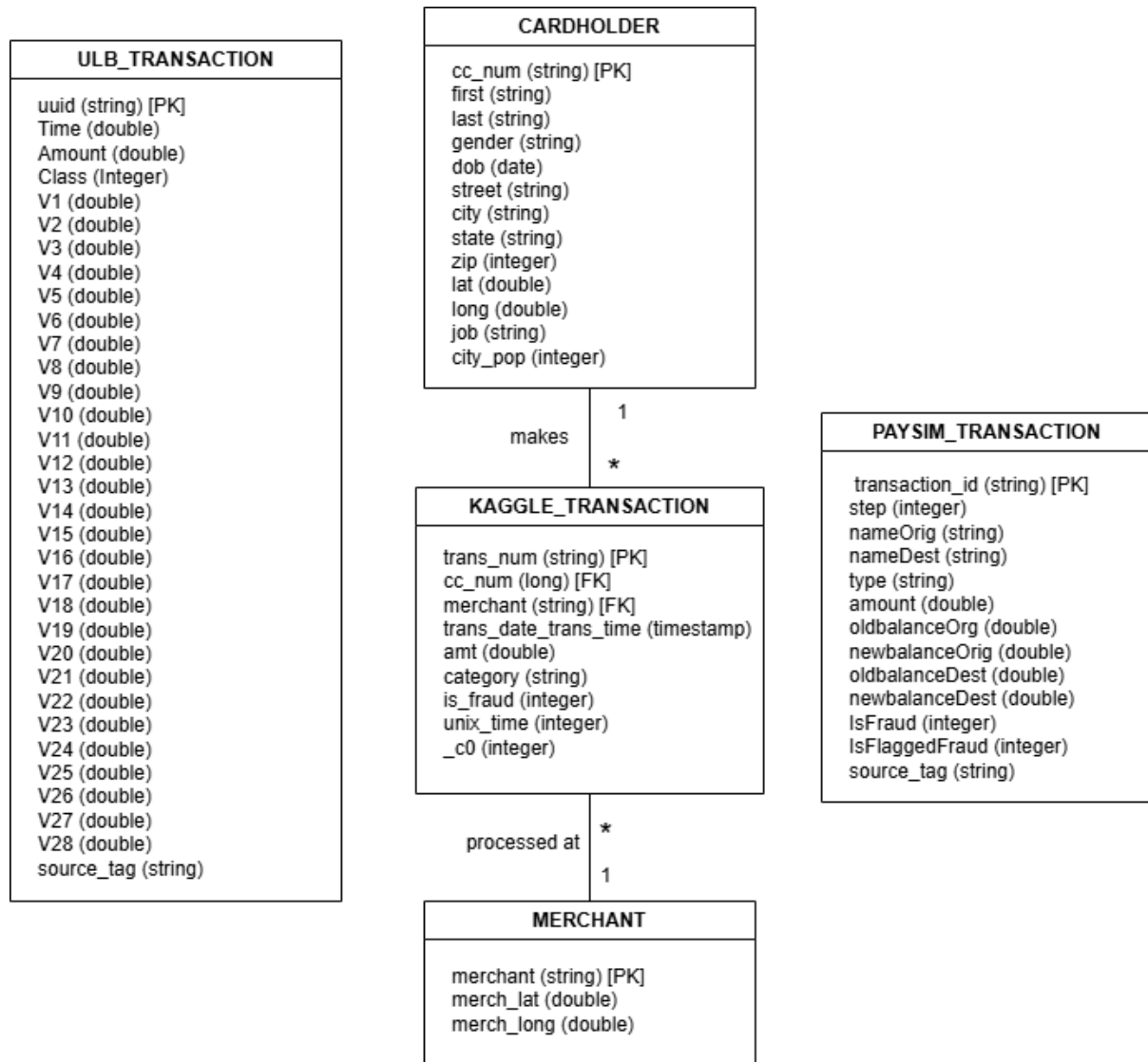


Figure 6: Fraud Detection Pipeline ERD Diagram

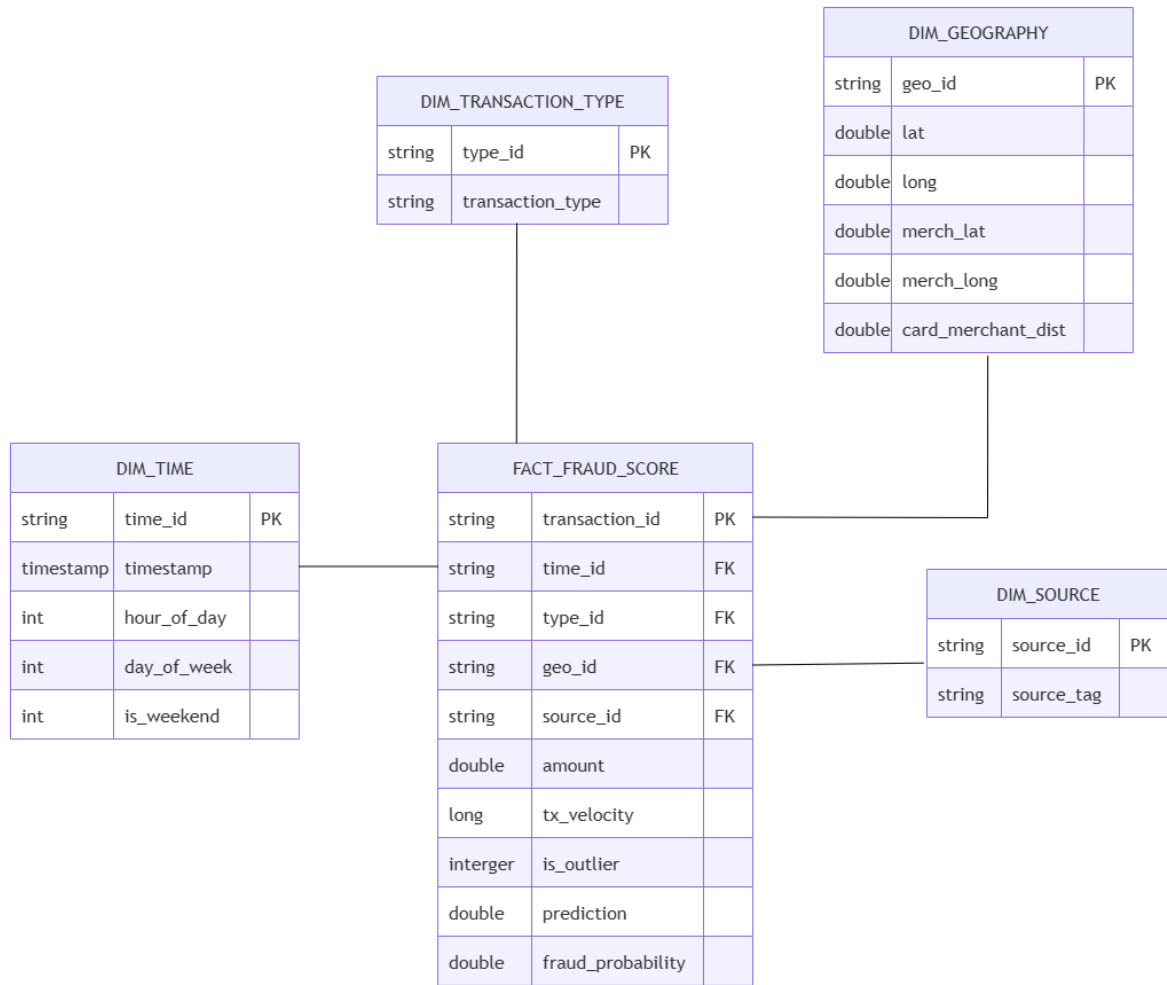


Figure 7: Fraud Detection Pipeline Multidimensional Star Schema Diagram

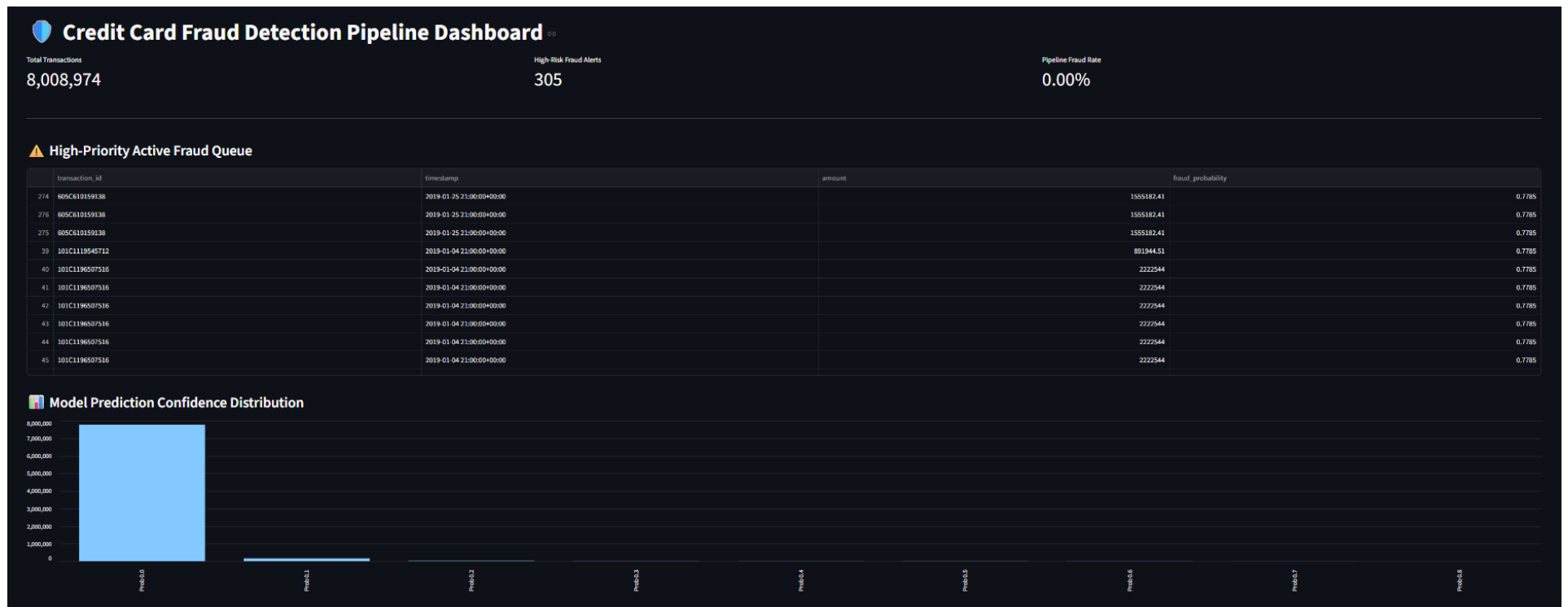


Figure 8: Deployed Dashboard Interface

```
=== Model Evaluation Results ===
Precision      : 0.9918
Recall         : 0.9921
F1-Score       : 0.9896
AUC-ROC        : 0.9530
```

Figure 9: Terminal Screenshot of Model Evaluation Results

```
-----
Scoring stage complete!
Total Records Processed : 8008974
High-Risk Alerts Flagged: 305
-----
```

Figure 10: Terminal Screenshot of Pipeline Scoring Result

```
=====
DATA PIPELINE INTEGRITY AUDIT REPORT
=====
Checking Bronze - Kaggle      ... [SUCCESS] - Version: 0 | Records: 1,296,675
Checking Bronze - ULB        ... [SUCCESS] - Version: 0 | Records: 284,807
Checking Bronze - PaySim      ... [SUCCESS] - Version: 0 | Records: 6,362,620
Checking Silver Layer         ... [SUCCESS] - Version: 0 | Records: 7,944,055
Checking Gold Layer           ... [SUCCESS] - Version: 0 | Records: 8,008,974
Checking Inference Scores     ... [SUCCESS] - Version: 0 | Records: 8,008,974
Checking Active Alert Queue   ... [SUCCESS] - Version: 0 | Records: 305
=====
Validation Metric: Total Scored records match dashboard limit? Yes
Validation Metric: High-Risk Alert anomalies isolated?           Yes
=====
STATUS: ALL DATA LAYERS VERIFIED & FULLY FUNCTIONAL (PASS)
=====
```

Figure 11: Terminal Screenshot of Pipeline Testing Result